PNU AI BOOSTER 2025

AI Booster
최종발표

Team AISFA | DrillMusic

김성욱
Team Leader / PM / App Dev

문경환
AI Researcher / AI Dev

전진혁
Cloud & Infra Specialist

RECAP

지난 발표 요약

2025.07 기준 — 바이브코딩의 한계

IDE 보조 도구에 불과
전반적 기능 개발 위임은 위험

인간 주도의 좋은 코드가 여전히 핵심

- 좋은 변수명, 적절한 문서화, 건전한 타입
- 높은 응집도, 작은 단위의 수정 가능한 코드
- 작은 context에서도 다룰 수 있는 구조

데이터 없는 도메인에서는 AI 활용 자체가 어려움

도전 선언

데이터가 없는 도메인에서
AI 활용에 도전하기로 결정

DrillMusic 프로젝트 착수

AI DJ 시스템 — 프로 DJ를 위한
AI 기반 트랙 추천 시스템 개발

BPM, Key, 에너지, 장르, 관객 반응을
동시에 고려하는 인지 부하를 줄여주는 도구

EXPERIMENT DESIGN

투 트랙 비교 실험

Human Driven

인간 주도 + AI 보조

도구: Cursor AI

인간 개발자가 설계·구현을 주도
AI는 코딩 보조 역할

비용: 인당 $20/월

3명 × 약 1.5개월 개발

VS

AI Driven

완전 AI 주도

도구: Claude Code

AI 에이전트가 설계·구현을 주도
인간은 방향 제시 및 검토

비용: 인당 $100/월

서브에이전트 / 에이전트 팀 활용

공통 목표: AI DJ 음악 추천 시스템 개발

Human Driven 결과

기술 스택

App Electron + React + TypeScript + Vite
AI CLAP 임베딩, FAISS 고속 유사도 검색
DB SQLite (로컬), Zustand (상태 관리)
연동 node-midi, chokidar (파일 감시)

추천 알고리즘

BPM/Key/에너지 호환성 필터링
CLAP 임베딩 기반 음질감 매칭
에너지 목표 (maintain/up/down/peak)
히스토리 기반 다양성 관리로 반복 방지

~1.5개월

개발 기간

3명

AI전문가 + 도메인전문가
+ 앱 개발자

31개

테스트 (100% 통과)

AI DRIVEN

AI Driven 결과

기술 스택

App Tauri v2 + React + TypeScript
DSP Rust (cpal, midir) — 오디오 처리
AI Python (Essentia, PyTorch, ONNX)
API FastAPI + PostgreSQL

아키텍처 & 주요 기능

Clean Architecture 4계층 구조
트랙 분석 / AI 추천 / AI 블렌드 / 라이브러리 임포트
MIDI 출력 / 커뮤니티 / 분석 대시보드
Rekordbox, Serato, Traktor 지원

~1시간

추천 알고리즘 개발

~2일

전체 시스템 개발
(실질 1일 미만)

823+

테스트 자동 작성

KEY INSIGHT

충격적 비교

	Human Driven	AI Driven
추천 알고리즘 음악 추천 로직 개발	**~1.5개월** 3명의 전문가 협업	**~1시간** 5~6개 에이전트 동시 협업
전체 시스템 앱 + AI + 백엔드 통합	**개발 중단** (알고리즘 집중) 앱 모듈 프로토타입 완성	**~2일** 실질 1일 미만 (토큰 한도 초과)

※ 추천 알고리즘 품질 또한 AI Driven이 더 만족스러운 결과

LESSON #1

써봐야 합니다

목격

5개의 에이전트가 동시에 소통하며 코드를 생성하는 광경을 목격

아무리 대체된다는 말을 들어도,
직접 보면 위기감이 다릅니다.

신입생부터 졸업생까지,
어쩌면 교수님들도.

방해

인간이 LLM의 작업 역량을 방해합니다

기존 철학을 유지하기 위해
권한을 넘기지 않았더니
진행이 안 됩니다.

검토가 불가능한 영역입니다.

전환

이해하려 하지 말고 잘 명령하고, 이해할 수 있는 내용을 요청하라

만들어진 코드를
읽기가 힘듭니다.
남이 써준 코드인데다가
인간 최적화가 아닙니다.

느껴봐야 합니다. 신입생부터 졸업생까지.

LESSON #2

준비해야 합니다

컴퓨터를 잘 쓰는 사람

- 어떤 기능이 있는지 잘 안다
- 필요한 프로그램을 다운받아 사용
- 단축키 등으로 빠르게 작업 완료

→

AI Native

- 어떤 작업이 가능한지 인지
- 필요한 서비스를 잘 골라 사용
- 스킬/플러그인으로 빠르게 완성

사람은 코드가 아닌 워크플로우를 생산합니다.

업무에 자주 사용하는 플러그인과 스킬을 준비해놓아야 AI를 잘 쓴다고 합니다.

LESSON #3

크게 봐야 합니다

실제 사례: 멀티 스택 통합 실패

AI가 스스로 기술 스택을 선정:

프론트: React / 앱 엔진: Rust / AI 로직: Python

부분 로직은 훌륭하게 작성했지만,
통합 구동에서 실패 — 이상한 방향으로 헤맴

시사점

작은 그림을 보는 사람은

이미 사라졌습니다.

AI는 부분은 잘 합니다.
전체를 설계하는 것은
인간의 역할입니다.

컴퓨터보다 계산이 빠르면
직접 계산하세요.

컴파일러보다 바이너리를
잘 짜면 직접 하세요.

**우리는 C를 씁니다.
이제 사람은 시스템을
설계할 줄 알아야 합니다.**

약 6개월 동안 세상이 바뀌었습니다.

무어의 법칙은 이제
CPU, 메모리를 넘어 AI로 왔습니다.

소프트웨어 3.0

하네스(Harness) — 우리가 갖춰야 할 능력

사고의 틀을 바꾸고 앞서가십시오.